

CO 3 - AT 3

1.code:

```
public class Main {
    static class SharedBuffer {
        private final int[] buffer;
        private int count = 0, front = 0, rear = 0;

        SharedBuffer(int size) {
            buffer = new int[size];
        }

        public synchronized void put(int item) {
            while (count == buffer.length) {
                try { wait(); } catch (InterruptedException e) {}
            }
            buffer[rear] = item;
            rear = (rear + 1) % buffer.length;
            count++;
            System.out.println("Produced: " + item + " (count=" + count + ")");
            notify();
        }

        public synchronized int get() {
            while (count == 0) {
                try { wait(); } catch (InterruptedException e) {}
            }
            int item = buffer[front];
            front = (front + 1) % buffer.length;
            count--;
            System.out.println("Consumed: " + item + " (count=" + count + ")");
            notify();
            return item;
        }
    }
}

static class ProducerThread extends Thread {
    private final SharedBuffer sb;
```

```

    ProducerThread(SharedBuffer sb) { this.sb = sb; }

    public void run() {
        for (int i = 1; i <= 10; i++) {
            sb.put(i);
            try { Thread.sleep(100); } catch (InterruptedException e) {}
        }
    }
}

static class ConsumerThread extends Thread {
    private final SharedBuffer sb;

    ConsumerThread(SharedBuffer sb) { this.sb = sb; }

    public void run() {
        for (int i = 1; i <= 10; i++) {
            sb.get();
            try { Thread.sleep(150); } catch (InterruptedException e) {}
        }
    }
}

public static void main(String[] args) {
    SharedBuffer sb = new SharedBuffer(3);
    System.out.println("Producer Consumer started");
    Thread p = new ProducerThread(sb);
    Thread c = new ConsumerThread(sb);
    p.start();
    c.start();
}
}

```

Google Chrome isn't your default browser [Set as default](#)

SIMATS ENGINEERING **SIMATS Online Programming Lab** **JAVA PROGRAMMING** **MATTA SWARNA AMBICA** 192472390RA

25 marks

Q4 Online Examination Thread Synchronization 25 marks

4/4 answered

Your Code

```

1 public class Main {
2     static class SharedBuffer {
3         private final int[] buffer;
4         private int count = 0, front = 0, rear = 0;
5
6         SharedBuffer(int size) {
7             buffer = new int[size];
8         }
9
10        public synchronized void put(int item) {
11            while (count == buffer.length) {
12                try { wait(); } catch (InterruptedException e) {}
13            }
14            buffer[rear] = item;
15            rear = (rear + 1) % buffer.length;
16            count++;
17            System.out.println("Produced: " + item + " (count=" + count);
18            notify();
19        }
20
21        public synchronized int get() {
22            while (count == 0) {
23                try { wait(); } catch (InterruptedException e) {}
24            }

```

Input

stdin input

Output

```

Producer Consumer started
Produced: 1 (count=1)
Consumed: 1 (count=0)
Produced: 2 (count=1)
Consumed: 2 (count=0)
Produced: 3 (count=1)

```

2.code :

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
class PrinterQueue {
```

```
    private final Queue<String> queue = new LinkedList<>();
```

```
    private final int CAPACITY = 3;
```

```
    // Synchronized method for users to submit documents (Producer)
```

```
    public synchronized void submitDocument(String doc, String userName) {
```

```
        while (queue.size() == CAPACITY) {
```

```
            try {
```

```
                System.out.println("[Wait] " + userName + " is waiting to submit: " + doc + "
```

```
(Queue Full)");
```

```
                wait(); // Wait until the printer processes a document
```

```
            } catch (InterruptedException e) {
```

```
                Thread.currentThread().interrupt();
```

```
            }
```

```
        }
```

```
        queue.add(doc);
```

```
        System.out.println("[Submitted] " + userName + " added: " + doc);
```

```
        notifyAll(); // Notify the printer that a new document is available
```

```
    }
```

```

// Synchronized method for the printer to process documents (Consumer)
public synchronized void printDocument() {
    while (queue.isEmpty()) {
        try {
            System.out.println("[Wait] Printer is idle... (Queue Empty)");
            wait(); // Wait until a user submits a document
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    String doc = queue.poll();
    System.out.println("[Printed] Printer processed: " + doc);
    notifyAll(); // Notify waiting users that space is now available in the queue
}
}

class UserThread extends Thread {
    private PrinterQueue printerQueue;
    private String userName;
    private String[] documents;

    public UserThread(PrinterQueue printerQueue, String userName, String[] documents)
    {
        this.printerQueue = printerQueue;
        this.userName = userName;
        this.documents = documents;
    }

    @Override
    public void run() {
        for (String doc : documents) {
            printerQueue.submitDocument(doc, userName);
            try {
                Thread.sleep(500); // Simulate time between submissions
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

```

```
    }  
    }  
}
```

```
class PrinterThread extends Thread {  
    private PrinterQueue printerQueue;  
  
    public PrinterThread(PrinterQueue printerQueue) {  
        this.printerQueue = printerQueue;  
    }  
  
    @Override  
    public void run() {  
        // Simulating continuous printing  
        for (int i = 0; i < 5; i++) {  
            printerQueue.printDocument();  
            try {  
                Thread.sleep(1000); // Simulate time taken to print  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        PrinterQueue sharedQueue = new PrinterQueue();  
  
        // Creating User Threads (Producers)  
        UserThread user1 = new UserThread(sharedQueue, "User-1", new  
String[] {"Report.pdf", "Resume.docx", "Data.csv"});  
        UserThread user2 = new UserThread(sharedQueue, "User-2", new  
String[] {"Photo.jpg"});  
  
        // Creating Printer Thread (Consumer)  
        PrinterThread printer = new PrinterThread(sharedQueue);
```

```

// Starting the threads
printer.start();
user1.start();
user2.start();
}
}

```

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS Engineering logo, the lab name, and the user's name (MATTIA SWARNA AMBICA) with a dropdown arrow. Below the header, there's a sidebar on the left with a question card for 'Q4 Online Examination' on 'Thread Synchronization' worth 25 marks, indicating '4/4 answered'. The main area is divided into three sections: 'Your Code' with a Java code editor, 'Input' with a text field containing 'stdin input', and 'Output' with a scrollable area showing the program's execution output.

```

1 import java.util.LinkedList;
2 import java.util.Queue;
3 public class Main {
4     private static final Queue<String> queue = new LinkedList<>();
5     private static final int CAPACITY = 3;
6     public static void submit(String doc, String userName) {
7         if (queue.size() >= CAPACITY) {
8             System.out.println("Wait] " + userName + " is waiting to ");
9             return;
10        }
11        queue.add(doc);
12        System.out.println("[Submitted] " + userName + " added: " + doc);
13    }
14
15    public static void print() {
16        if (queue.isEmpty()) {
17            System.out.println("Wait] Printer is idle... (Queue Empty)");
18            return;
19        }
20        String doc = queue.poll();
21        System.out.println("[Printed] Printer processed: " + doc);
22    }
23
24    public static void main(String[] args) {

```

Output:

```

[Submitted] User-1 added:
Report.pdf
[Submitted] User-1 added:
Resume.docx
[Submitted] User-2 added:
Photo.jpg

```

3. Code:

```

public class Main {
    static class SharedBuffer {
        private final int[] buffer;
        private int count = 0;
        private int front = 0;
        private int rear = 0;

        public SharedBuffer(int size) {
            buffer = new int[size];
        }

        public synchronized void put(int item) {
            while (count == buffer.length) {
                try {
                    wait();
                } catch (InterruptedException e) {

```

```

        System.out.println("Producer interrupted.");
    }
}
buffer[rear] = item;
rear = (rear + 1) % buffer.length;
count++;
System.out.println("Producer produced: " + item + " | Buffer count = " + count);
notify();
}

public synchronized int get() {
    while (count == 0) {
        try {
            wait();
        } catch (InterruptedException e) {
            System.out.println("Consumer interrupted.");
        }
    }
    int item = buffer[front];
    front = (front + 1) % buffer.length;
    count--;
    System.out.println("Consumer consumed: " + item + " | Buffer count = " + count);
    notify();
    return item;
}
}

static class ProducerThread extends Thread {
    private final SharedBuffer sharedBuffer;
    private final int produceCount;

    public ProducerThread(SharedBuffer sharedBuffer, int produceCount) {
        this.sharedBuffer = sharedBuffer;
        this.produceCount = produceCount;
    }

    public void run() {

```

```

        for (int i = 1; i <= produceCount; i++) {
            sharedBuffer.put(i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println("Producer sleep interrupted.");
            }
        }
    }
}

```

```

static class ConsumerThread extends Thread {
    private final SharedBuffer sharedBuffer;
    private final int consumeCount;

    public ConsumerThread(SharedBuffer sharedBuffer, int consumeCount) {
        this.sharedBuffer = sharedBuffer;
        this.consumeCount = consumeCount;
    }

    public void run() {
        for (int i = 1; i <= consumeCount; i++) {
            sharedBuffer.get();
            try {
                Thread.sleep(150);
            } catch (InterruptedException e) {
                System.out.println("Consumer sleep interrupted.");
            }
        }
    }
}

```

```

public static void main(String[] args) {
    int bufferSize = 3;
    int totalItems = 10;
    SharedBuffer sharedBuffer = new SharedBuffer(bufferSize);
    Thread producer = new ProducerThread(sharedBuffer, totalItems);
}

```



```

Thread consumer = new ConsumerThread(sharedBuffer, totalItems);
System.out.println("Producer Consumer started");
producer.start();
consumer.start();
try {
    producer.join();
    consumer.join();
} catch (InterruptedException e) {
    System.out.println("Main thread interrupted.");
}
System.out.println("\nProducer Consumer finished.");
}
}

```

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS Engineering logo, the lab name, and a user profile for MATTA SWARNA AMBICA. The main area is divided into three sections: a sidebar on the left with a question card (Q4: Online Examination - Thread Synchronization, 25 marks), a central code editor, and a right panel for input and output.

Your Code

```

1 public class Main {
2     static class SharedBuffer {
3         private final int[] buffer;
4         private int count = 0;
5         private int front = 0;
6         private int rear = 0;
7
8         public SharedBuffer(int size) {
9             buffer = new int[size];
10        }
11
12        public synchronized void put(int item) {
13            while (count == buffer.length) {
14                try {
15                    wait();
16                } catch (InterruptedException e) {
17                    System.out.println("Producer interrupted.");
18                }
19            }
20            buffer[rear] = item;
21            rear = (rear + 1) % buffer.length;
22            count++;
23            System.out.println("Producer produced: " + item + " | Buffer
24            notify();
25        }

```

Input

stdin input

Output

```

Producer Consumer started
Producer produced: 1 | Buffer
count = 1
Consumer consumed: 1 | Buffer
count = 0
Producer produced: 2 | Buffer

```

4. Code :

```

public class Main {
    private int submissionCounter = 0;

    public synchronized void submitAnswer(String studentName) {
        System.out.println(studentName + " is submitting answer");
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
        }
    }
}

```

```
        submissionCounter++;
        System.out.println(studentName + " submitted successfully. Total submissions = " +
submissionCounter);
    }
```

```
public int getFinalCount() {
    return submissionCounter;
}
```

```
static class StudentThread extends Thread {
    private final Main system;
    private final String studentName;
    public StudentThread(Main system, String studentName) {
        this.system = system;
        this.studentName = studentName;
    }
    public void run() {
        system.submitAnswer(studentName);
    }
}
```

```
public static void main(String[] args) {
    Main examSystem = new Main();
    Thread s1 = new StudentThread(examSystem, "Student_A");
    Thread s2 = new StudentThread(examSystem, "Student_B");
    Thread s3 = new StudentThread(examSystem, "Student_C");
    Thread s4 = new StudentThread(examSystem, "Student_D");
    Thread s5 = new StudentThread(examSystem, "Student_E");
    s1.start();
    s2.start();
    s3.start();
    s4.start();
    s5.start();
    try {
        s1.join();
        s2.join();
        s3.join();
    }
```

```
s4.join();
s5.join();
} catch (InterruptedException e) {
    System.out.println("Main thread interrupted.");
}
System.out.println("\nExamination Finished ");
System.out.println("Final number of total submissions = " +
examSystem.getFinalCount());
}
}
```